IAM-08: Multi-Environment IAM

> **Series:** IAM — IAM Administration | **Notebook:** 8 of 12 | **Created:** January 2026 | **Last Updated:** 10/12/2026

Scaling IAM Across Multiple Environments

Enterprise organizations typically have multiple Dynatrace environments (development, staging, production). This notebook covers strategies for managing IAM consistently across environments while maintaining security boundaries.

Table of Contents

1. Multi-Environment Architecture
2. Account-Level vs Environment-Level Permissions
3. Centralized vs Federated Models
4. Cross-Environment Access Patterns
5. Break-Glass Procedures
6. Synchronizing IAM Configuration
7. Multi-Tenant Considerations
8. Monitoring Multi-Environment IAM

Prerequisites

Requirement	Details
Dynatrace Environments	Multiple environments under one account

Requirement	Details
Permissions	Account-level IAM admin
Knowledge	IAM-01 through 07 completed

1. Multi-Environment Architecture

Understanding the relationship between accounts and environments is foundational.

Account and Environment Hierarchy

```
Dynatrace Account (Organization Level)
├── Environment: Production
│   ├── Entities, Logs, Traces
│   └── Environment-specific settings
├── Environment: Staging
│   ├── Entities, Logs, Traces
│   └── Environment-specific settings
└── Environment: Development
    ├── Entities, Logs, Traces
    └── Environment-specific settings
```

Permission Scopes

Scope	What It Controls	Example
Account	Users, groups, SSO, billing	Create users, manage IdP
Environment	Data access, settings, configurations	Read logs, modify alerting

Common Environment Patterns

Pattern	Environments	Use Case
Standard	Dev, Stage, Prod	Software development
Regional	US, EU, APAC	Geographic compliance
BU-based	Retail, Finance, HR	Business unit isolation

Pattern	Environments	Use Case
Tenant	Customer A, B, C	MSP/multi-tenant

2. Account-Level vs Environment-Level Permissions

Understanding where permissions are managed is critical for proper access control.

Account-Level Permissions

Managed in Account Management, apply across all environments:

Permission	Description
`account-user-management`	View and manage users, groups and policies
`account-viewer`	View account information
`account-company-info`	Manage account and company settings

Environment-Level Permissions

Scoped to specific environments:

Permission	Description
`storage:logs:read`	Read logs in environment
`storage:metrics:read`	Read metrics
`settings:objects:read`	Read configuration
`settings:objects:write`	Modify configuration

Permission Assignment Matrix

Role	Account Perms	Prod	Stage	Dev
IAM Admin	`account-user-management`	-	-	-

Role	Account Perms	Prod	Stage	Dev
Prod Viewer	-	Read	-	-
Prod Editor	-	Read/Write	-	-
Dev Team	-	Read	Read/Write	Full
Platform Team	-	Full	Full	Full

3. Centralized vs Federated Models

Choose a governance model based on organization structure.

Centralized Model

Single IAM team manages all access.

Aspect	Approach
Group creation	Central team only
Policy authoring	Central team only
Access requests	Ticketing system
Approvals	Central team

Pros:
- Consistent standards
- Single point of control
- Easier compliance

Cons:
- Bottleneck for requests
- Slower response time
- Less team autonomy

Federated Model

Delegated administration to teams.

Aspect	Approach
Group creation	Team leads (within namespace)
Policy authoring	Central team defines templates
Access requests	Direct to team lead
Approvals	Team lead + central audit

Pros:
- Faster access provisioning
- Team ownership
- Scales with organization

Cons:
- Requires governance framework
- Risk of inconsistency
- More complex auditing

Hybrid Model (Recommended)

Responsibility	Owner
Account-level IAM	Central team
Production access	Central team
Non-prod access	Team leads
Policy templates	Central team
Custom policies	Team leads (with approval)

4. Cross-Environment Access Patterns

Define how users access multiple environments.

Pattern 1: Environment-Specific Groups

Separate groups per environment.

Group	Environment	Permissions
`dt-prod-viewers`	Production	Read-only
`dt-stage-editors`	Staging	Read/Write
`dt-dev-admins`	Development	Full

Use when: Strict environment isolation required.

Pattern 2: Role-Based Groups

Same role, different environment permissions.

Group	Prod	Stage	Dev
`dt-sre-team`	Editor	Editor	Editor
`dt-dev-team`	Viewer	Editor	Admin
`dt-security-team`	Viewer	Viewer	Viewer

Use when: Consistent role across environments with varying access levels.

Pattern 3: Escalation-Based Access

Production access requires explicit escalation.

Group	Normal Access	Escalated Access
`dt-oncall`	Stage/Dev	Production (during incident)
`dt-deploy`	Stage	Production (during deploy window)

Use when: Production access is exception, not default.

5. Break-Glass Procedures

Emergency access when normal procedures can't be followed.

What is Break-Glass?

Emergency access mechanism for critical incidents when:
- Normal approvers unavailable
- SSO/IdP is down
- Critical incident requires immediate elevated access

Break-Glass Components

Component	Purpose
Emergency account	Local account (not SSO)
Elevated permissions	Admin or full access
Secure storage	Password vault, split secrets
Audit logging	All actions logged
Post-incident review	Mandatory after use

Break-Glass Setup

1. **Create emergency account**
   - Local Dynatrace account (not SSO-dependent)
   - Strong, complex password
   - MFA with hardware token
2. **Assign minimal necessary permissions**
   - Typically `account-user-management` + the Admin User default policy
   - Or scoped to specific emergency needs
3. **Secure the credentials**
   - Store in enterprise vault (CyberArk, HashiCorp)
   - Or use split-knowledge (2+ people needed)
   - Document retrieval procedure
4. **Define usage criteria**
   - When break-glass can be used
   - Who can authorize
   - Notification requirements

5. **Establish post-incident process**
   - Rotate credentials after each use
   - Review all actions taken
   - Document in incident report

Break-Glass Policy Template

```
BREAK-GLASS ACCESS POLICY

Authorized Situations:
☐ P1/P2 incident AND normal approvers unavailable
☐ SSO/IdP outage preventing authentication
☐ Security incident requiring immediate response

NOT Authorized:
☐ Convenience (approver busy but reachable)
☐ Non-urgent requests
☐ Bypassing normal approval process

Usage Steps:
1. Attempt normal access/approval first
2. Document why break-glass is needed
3. Notify security team
4. Retrieve credentials from vault
5. Perform necessary actions only
6. Log out immediately when done
7. Notify IAM team for credential rotation
8. Complete post-incident review within 24h
```

6. Synchronizing IAM Configuration

Keep IAM configuration consistent across environments.

Configuration as Code

Store IAM configuration in version control:

```
iam-config/
├── groups/
│   ├── platform-team.yaml
│   ├── dev-team.yaml
│   └── security-team.yaml
├── policies/
│   ├── standard-user.yaml
│   ├── pro-user.yaml
│   └── log-reader.yaml
└── boundaries/
    ├── production-boundary.yaml
    └── sensitive-data-boundary.yaml
```

Synchronization Approaches

Approach	Pros	Cons
Manual	Simple	Error-prone, doesn't scale
Terraform	Version controlled, repeatable	Learning curve
Dynatrace API	Direct control	Custom scripting needed
Monaco	Dynatrace-specific	Configuration complexity

Change Management Process

Step	Action	Approval
1	Create PR with IAM change	Developer
2	Review configuration	IAM team
3	Apply to dev environment	Automated
4	Test in dev	Developer
5	Apply to staging	Automated
6	Validate in staging	IAM team
7	Apply to production	Manual approval + deploy

7. Multi-Tenant Considerations

Special considerations for MSPs and multi-tenant deployments.

Multi-Tenant Architecture

Model	Isolation Level	Use Case
Separate Accounts	Highest	Different legal entities
Separate Environments	High	Business unit isolation
Boundaries within Environment	Medium	Team isolation
Segments only	Lower	Data filtering only

Tenant Isolation Checklist

```
☐ Data cannot be accessed across tenants
☐ Users cannot see other tenants exist
☐ Configuration cannot impact other tenants
☐ Audit logs are tenant-specific
☐ Billing is tenant-specific
```

MSP Considerations

Aspect	Recommendation
Customer environments	Separate environment per customer
MSP admin access	Dedicated MSP admin group
Customer admin access	Scoped to their environment only
Audit	Separate audit logs per customer
Compliance	Document isolation in SOC2

Monitoring Multi-Environment IAM

Query patterns for multi-environment oversight. Run these in each environment — audit events are recorded per environment, so there is no cross-environment audit query. Account-level changes are in the account audit log (Account Management > Settings > Audit log).

```
// Who calls this environment, from where (by organization, provider, source)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| summarize requests = count(), by:{user.organization, event.provider,
request.source}
| sort requests desc
| limit 25
```

```
// Write activity in this environment by organization — account-level changes
are in Account Management &gt; Settings &gt; Audit log
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
// Corrected 10/12/2026: many POSTs change nothing. DQL queries, permission
checks, verify/autocomplete/preview
// calls and classic token lookups are reads; app-state saves are UI
bookkeeping; workflow runs (…/run) are
// executions, not edits; /lookups/ uploads are a scheduled job. Over 30 days
on the validation
// tenant these were 177,504 of 198,069 write-verb events (90 %). Check the
remaining resource
// list on your tenant for further read-style POSTs.
| filter not(contains(resource, "/query:") or contains(resource, "effective-
permissions")
             or endsWith(resource, "/lookup") or contains(resource, "/verify")
             or contains(resource, "/autocomplete") or contains(resource,
"filter-preview")
             or endsWith(resource, "/run") or startsWith(resource,
"/platform/state/v1/user-app-states/")
             or contains(resource, "lookup:upload") or startsWith(resource,
"/lookups/"))
| summarize actions = count(), by:{user.organization, event.type}
| sort actions desc
```

```
// Break-glass / emergency account usage — rare-actor detection
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" — the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| summarize {events = count(), first_seen = takeMin(timestamp), last_seen =
takeMax(timestamp)}, by:{user.id}
| filter events &lt; 10
| sort events asc
| limit 25
```

Next Steps

With multi-environment IAM understood, proceed to troubleshooting:

Recommended Path

1. **IAM-09: Troubleshooting Access Issues** - Debug permission problems systematically

Multi-Environment Checklist

Before moving on, ensure you have:

- [ ] Documented all environments and their purposes
- [ ] Defined governance model (centralized, federated, hybrid)
- [ ] Created environment-specific access groups
- [ ] Established break-glass procedure
- [ ] Implemented IAM configuration as code
- [ ] Documented multi-tenant isolation (if applicable)

Summary

In this notebook, you learned:

- Multi-environment architecture and hierarchy
- Account-level vs environment-level permissions
- Centralized, federated, and hybrid governance models
- Cross-environment access patterns
- Break-glass procedures for emergency access
- IAM configuration synchronization approaches
- Multi-tenant considerations for MSPs

References

- Account Management (DT docs)
- Dynatrace Terraform Provider

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.